Here is the revised documentation with all three changes woven in throughout.

PDF/Word to BPMN Extraction Pipeline — Technical Specification

Version: 1.1-draft **Status:** Ready for Developer Review / PM Sign-off **Changelog from v1.0:** L1 migrated to Docling; pipeline philosophy shifted to LLM-primary with rules as structural scaffolding only; LLM call budget revised to 5–8 calls per document.

Table of Contents

1. System Overview
2. Core Data Schemas
3. Orchestration & Job State
4. L1 — Document Extraction
5. L2 — Segmentation
6. L3 — Block Classifier
7. L4 — Context Indexing
8. L5 — Block Enrichment
9. L6 — Block Atomizer
10. L7 — Node Detector
11. L8 — Edge Detector
12. L9 — DAG Resolver
13. L10 — BPMN Translation
14. LLM Integration Guide
15. Validation Gates Reference
16. Open Questions for PM Sign-off

1. System Overview

Purpose

Convert unstructured process documents (SOPs, procedure manuals) into valid, structured BPMN 2.0 graphs through a 10-layer sequential enrichment pipeline.

Design Philosophy

SOPs are inherently complex, inconsistent, and written in natural language. Rule-based systems alone are brittle against the real-world variation in how process documents are authored — heading styles differ, actor names are abbreviated, conditions are buried in prose, and steps are often compound. This pipeline treats **LLM as the primary intelligence** at every semantically complex layer. Rules and structural heuristics serve a narrower purpose: to provide reliable scaffolding for things that are deterministic by nature (file parsing, tree construction, ID generation, layout math) and to batch/filter LLM inputs efficiently.

Where rules are appropriate:

- Structural parsing: splitting a document on headings, identifying list depth, assigning IDs
- Cheap pre-filters: skipping LLM calls for blocks that are obviously headers or blank
- Post-LLM validation: asserting that LLM output conforms to required schema

Where LLM is primary:

- Any classification, interpretation, or resolution that depends on natural language meaning
- Actor deduplication, cross-reference resolution, pronoun resolution, block classification, atomization

LLM Call Budget

The pipeline is designed for **5–8 LLM calls per document**. SOPs are complex enough that attempting to replace any of these with heuristics will produce meaningfully worse results. The calls are batched at the layer level — all blocks for a given layer are processed in one or two calls, not one call per block.

Lay	Call	Purpose	Batch strategy
L3	Call 1	Classify all non-trivially-structural blocks	All blocks in one prompt
L4	Call 2	Deduplicate and canonicalize actor list	Full actor candidate list in one prompt
L4	Call 3 (optional)	Extract glossary from definitions section	Full definitions section in one prompt
L5	Call 4	Enrich all blocks: resolve pronouns, cross-refs, conditions	All blocks requiring enrichment in one prompt
L6	Call 5–6	Atomize all STEP/DECISION/ EXCEPTION blocks	Batched by section, 1–2 calls depending on document length
L7–L8	Call 7	Detect gateway types and infer non-obvious edges	All decision blocks + cross-ref blocks in one prompt

Total: 5–7 calls for typical documents. Up to 8 for long or complex SOPs where L6 splits into 2 batches.

Architecture Summary

```
Document File (PDF or Word)
    ↓
[L1]  Extraction via Docling   → Structured Markdown +
document JSON
    ↓
[L2]  Segmentation             → Document tree of Block
objects
    ↓
[L3]  Block Classifier         → Typed blocks  [LLM Call 1]
    ↓
[L4]  Context Indexing         → ContextIndex  [LLM Calls 2–
3]
    ↓
[L5]  Block Enrichment         → Enriched blocks  [LLM Call
4]
```

```
    ↓
[L6]  Block Atomizer           → AtomicUnit[]  [LLM Calls 5–
6]
    ↓
[L7]  Node Detector            → BPMNNode[]
    ↓
[L8]  Edge Detector            → BPMNEdge[]  [LLM Call 7]
    ↓
[L9]  DAG Resolver             → Validated BPMN graph
    ↓
[L10] BPMN Translator          → BPMN 2.0 XML file
```

Key Principles

- **Each layer is independently retriable.** All writes are idempotent — re-running a layer on the same input produces the same output.
- **Failures are structured.** Every layer writes a typed error payload to the job record before terminating, never silently continues with bad data.
- **LLM output is always schema-validated.** Every LLM call uses structured output mode. A response that does not conform to the required JSON schema is retried once, then flagged for review.
- **Validation gates are non-negotiable.** Every layer boundary has a minimum quality assertion. Failing the gate terminates the job with a clear reason.

2. Core Data Schemas

2.1 Block

```
interface Block {
  block_id: string;
  job_id: string;
  parent_id: string | null;
  children_ids: string[];

  // Populated in L2
  raw_text: string;
  heading_path: string[];
  page_number: number | null;
  list_depth: number;
  list_index: string | null;

  // Populated in L3
  block_type: BlockType | null;
  block_type_confidence: number | null;
  block_type_method: "structural_skip" | "llm" | null;
  // Note: "structural_skip" replaces "rules" — only used for
HEADER blocks
```

```
  // and blank blocks that don't need LLM classification.
  // Everything else goes through LLM.

  // Populated in L5
  resolved_actor: string | null;
  condition_scope: string | null;
  cross_refs: CrossRef[];
  pronoun_resolution: PronounResolution | null;
  enrichment_version: number;

  // Populated in L6
  atomic_units: AtomicUnit[];

  needs_review: boolean;
  review_reasons: string[];
}

type BlockType =
  | "STEP"
  | "DECISION"
  | "EXCEPTION"
  | "ACTOR"
  | "CONDITION"
  | "NOTE"
  | "HEADER"
  | "UNKNOWN";

interface CrossRef {
  ref_text: string;
  resolved_block_id: string | null;
  resolution_method: "structural_anchor" | "llm" |
"unresolved";
}

interface PronounResolution {
  original_pronoun: string;
  resolved_to: string;
  confidence: number;
  method: "llm";
}
```

2.2 AtomicUnit

```
interface AtomicUnit {
  unit_id: string;
  block_id: string;
```

```
  sequence_in_block: number;

  action: string;        // REQUIRED. Verb-phrase.
  actor: string;         // REQUIRED. Canonical name from
actor registry.
  condition: string | null;
  output: string | null;

  is_terminal: boolean;
  is_start: boolean;
}
```

2.3 BPMNNode

```
interface BPMNNode {
  node_id: string;
  job_id: string;
  unit_id: string | null;

  bpmn_type: BPMNNodeType;
  label: string;
  actor: string | null;
  gateway_type: GatewayType | null;

  x: number | null;
  y: number | null;
  width: number | null;
  height: number | null;
}

type BPMNNodeType =
  | "START_EVENT"
  | "END_EVENT"
  | "TASK"
  | "GATEWAY"
  | "BOUNDARY_EVENT"
  | "SUBPROCESS";

type GatewayType = "XOR" | "AND" | "OR";
```

2.4 BPMNEdge

```
interface BPMNEdge {
  edge_id: string;
  job_id: string;
  source_node_id: string;
  target_node_id: string;
```

```
  label: string | null;
  is_default: boolean;
  edge_type: "SEQUENCE_FLOW" | "MESSAGE_FLOW" |
"ASSOCIATION";
}
```

2.5 ActorRegistry

```
interface ActorRegistry {
  job_id: string;
  actors: Actor[];
}

interface Actor {
  canonical_name: string;
  aliases: string[];
  source_section: string;
  source_method: "structural_extraction" | "llm_scan";
}
```

2.6 ContextIndex

```
interface ContextIndex {
  job_id: string;
  section_anchors: SectionAnchor[];
  glossary: GlossaryEntry[];
  exception_blocks: string[];
  actor_registry: ActorRegistry;
}

interface SectionAnchor {
  anchor_text: string;
  block_id: string;
  heading_path: string[];
}

interface GlossaryEntry {
  term: string;
  definition: string;
  block_id: string;
  definition_method: "structured_definition" | "llm";
}
```

3. Orchestration & Job State

3.1 Job Record

```
interface Job {
  job_id: string;
  source_file_path: string;
  status: JobStatus;
  current_layer: number | null;
  error: JobError | null;
  created_at: timestamp;
  updated_at: timestamp;
  layer_timestamps: Record<string, timestamp>;
  review_flags: ReviewFlag[];
  llm_call_log: LLMCallRecord[];
}

type JobStatus =
  | "PENDING"
  | "RUNNING"
  | "COMPLETE"
  | "FAILED"
  | "NEEDS_REVIEW";

interface JobError {
  layer: number;
  error_code: string;
  message: string;
  traceback: string | null;
}

interface ReviewFlag {
  block_id: string | null;
  layer: number;
  reason: string;
}

interface LLMCallRecord {
  layer: number;
  prompt_template: string;
  input_tokens: number;
  output_tokens: number;
  latency_ms: number;
  cached: boolean;
}
```

3.2 Orchestration Flow

```
for layer in [L1, L2, L3, L4, L5, L6, L7, L8, L9, L10]:
    job.status = RUNNING
```

```
    job.current_layer = layer.number

    try:
        result = layer.run(job_id)
    except LayerError as e:
        job.status = FAILED
        job.error = { layer, code, message, traceback }
        return

    gate_result = layer.validate_gate(job_id)
    if not gate_result.passed:
        job.status = FAILED
        job.error = { layer, code: "GATE_FAILURE", message:
gate_result.reason }
        return

job.status = COMPLETE if no review_flags else NEEDS_REVIEW
```

4. L1 — Document Extraction

Responsibility

Convert the source file into a normalized, structure-preserving Markdown string using Docling. Docling is the sole extraction engine — it handles both PDF and Word inputs and produces reliable heading hierarchies, table Markdown, and list preservation out of the box, without custom heuristics.

Why Docling

Docling provides a unified extraction API that handles the structural complexity of real-world documents (nested tables, mixed heading styles, multi-column layouts) far more reliably than custom PyMuPDF or python-docx code. It outputs both a Markdown string and a structured document JSON, both of which are stored and used downstream.

Inputs

Field	Type	Description
`source_file_path`	`string`	Absolute path to uploaded file
`file_extension`	`"pdf" | "docx" | "doc"`	Passed to Docling's format selector

Outputs

Field	Type	Description

`markdown`	`string`	Full document as Docling-produced Markdown
`docling_document`	`object`	Docling's native structured JSON representation (stored for potential downstream use in L2)
`page_count`	`number`	Total pages
`metadata`	`object`	Author, title, creation date from file metadata

Processing

```
from docling.document_converter import DocumentConverter

converter = DocumentConverter()
result = converter.convert(source_file_path)

markdown = result.document.export_to_markdown()
docling_doc = result.document.export_to_dict()
page_count = result.document.num_pages()
```

Docling handles both `.pdf` and `.docx` natively. For `.doc` files, convert to `.docx` first using LibreOffice headless before passing to Docling:

```
soffice --headless --convert-to docx document.doc
```

Post-processing (minimal):

Only two normalization steps are applied after Docling extraction, both structural not heuristic:

1. Collapse 3+ consecutive blank lines into 2
2. Strip page header/footer repetitions that Docling may not suppress (identify by identical lines appearing on ≥ 3 pages in the `docling_document` page objects)

No custom heading detection, no custom table flattening, no custom list parsing — Docling handles all of this.

Validation Gate

Check	Failure code	Action
`len(markdown.strip()) >=`	`L1_EMPTY_OUTPUT`	Fail job
Docling conversion completes without exception	`L1_CONVERSION_FAILED`	Fail job with Docling error message

5. L2 — Segmentation

Responsibility

Parse the Docling-produced Markdown into a hierarchical tree of `Block` objects. Assign a high-level SOP class to the document.

This is the one layer that remains fully structural and deterministic — the input is already clean Markdown from Docling, and splitting on headings and list markers is a parsing problem, not a language understanding problem.

Inputs

Field	Type	Description
`markdown`	`string`	L1 output
`docling_document`	`object`	Docling structured JSON (used for page number attribution per block)

Outputs

- Persisted tree of `Block` objects (structural fields only; `block_type` is null at this stage)
- `sop_class` annotation on the job record

SOP Class Taxonomy

SOP Class	Detection keywords
`HR_PROCESS`	employee, onboarding, offboarding, leave, payroll, appraisal, termination
`IT_PROCESS`	access, provisioning, ticket, incident, server, deployment, credentials
`FINANCE_PROCESS`	invoice, expense, approval, budget, reimbursement, purchase order
`COMPLIANCE_PROCESS`	audit, regulatory, risk, policy, violation, breach
`OPERATIONS_PROCESS`	production, quality, inventory, dispatch, warehouse, shipment
`GENERIC_PROCESS`	Fallback

Detection: keyword frequency scan across the full Markdown. The class with the most hits wins. Ties go to first match in table order.

Processing

Step 1 — Split on headings

Iterate the Markdown line by line. A heading line matches `^#{1,6}\s`. Create a `HEADER` Block for each heading. Maintain `heading_path` as a stack updated on each heading encountered.

Step 2 — Assign content to headings

All non-heading lines between two headings are children of the preceding heading block.

Step 3 — Parse list items

Within each heading's content, split on list markers (`^\s*[-*]\s` or `^\s*\d+[.)]\s`). Each list item becomes a child Block. `list_depth` is derived from leading whitespace (every 2-space indent = +1 depth). `list_index` is the marker character(s).

Step 4 — Paragraph blocks

Contiguous non-list, non-heading lines become a single paragraph Block with `list_depth = 0`.

Step 5 — Page number attribution

For each Block, look up the corresponding page range in `docling_document` using character offset or heading match. Set `block.page_number`.

Step 6 — SOP class assignment

Keyword frequency scan. Set `sop_class` on job record.

Validation Gate

Check	Failure code	Action
Tree has at least 1 non-root node	`L2_EMPTY_TREE`	Fail job
At least 1 block with `list_depth >= 1` or paragraph content	`L2_NO_CONTENT`	Fail job

6. L3 — Block Classifier

Responsibility

Assign a `BlockType` to every non-HEADER block in the document tree.

Design Decision

Classification is primarily LLM-driven. Rule-based pre-filtering only skips blocks that require zero language understanding to classify: `HEADER` blocks (already typed in L2) and blank blocks. Everything else — including blocks that seem "obviously" a step — goes to the LLM. This is deliberate: heuristics for classifying natural language blocks (imperative verb lists, keyword triggers) are both brittle and domain-specific. The LLM handles variation in phrasing, passive voice, abbreviated instructions, and domain terminology naturally.

Inputs

- Job's block tree (L2 output)

Outputs

- Every non-HEADER, non-blank Block has `block_type`, `block_type_confidence`, `block_type_method` set

Processing

Step 1 — Structural pre-filter (no LLM)

Mark the following blocks as classified without LLM, set `block_type_method = "structural_skip"`:

- Blocks already typed `HEADER` in L2 → skip
- Blocks where `raw_text.strip() == ""` → set `block_type = "NOTE"`, confidence 1.0, skip

All remaining blocks proceed to LLM classification.

Step 2 — Batch LLM Classification (LLM Call 1)

Collect all unclassified blocks. Send them all in a single prompt using the `CLASSIFY_BLOCKS_BATCH` template (see Section 14). The LLM receives the full list of blocks with their context (heading path, list depth, list index, raw text) and returns a classification for each.

Context provided per block in the batch:

- `heading_path`: gives the LLM section context
- `list_depth` and `list_index`: helps distinguish a numbered step from a sub-condition
- `raw_text`: the content itself
- `preceding_block_text`: the immediately preceding block's text (helps resolve ambiguity for short blocks)

The LLM returns a JSON array with one entry per block: `{ block_id, block_type, confidence, reasoning }`.

Step 3 — Apply results

For each returned classification, set `block_type`, `block_type_confidence`, `block_type_method = "llm"` on the corresponding Block. If any block_id is missing from the LLM response, mark it `UNKNOWN` with confidence 0.

Validation Gate

Check	Failure code	Action
At least 1 block with `block_type == "STEP"`	`L3_NO_STEPS_FOUND`	Fail job
UNKNOWN rate < 20%	`L3_HIGH_UNKNOWN_RATE`	Set job `status = NEEDS_REVIEW`; continue

7. L4 — Context Indexing

Responsibility

Build the `ContextIndex`: section anchors (structural), actor registry (LLM-deduped), glossary (LLM-extracted), and exception block index (structural).

Inputs

- Block tree with `block_type` populated (L3 output)

Outputs

- Persisted `ContextIndex` object

Processing

4a — Section Anchors (structural, no LLM)

For every `HEADER` block, create a `SectionAnchor` mapping its heading text to `block_id`. Also create anchors for shortened references (e.g. heading "3.2 Approval Process" gets anchors for "Section 3.2", "3.2", and "Approval Process").

For every numbered `STEP` block at `list_depth == 1`, create an anchor for `"Step {list_index}"` → `block_id`.

4b — Exception Block Index (structural, no LLM)

Collect all `block_id`s of blocks with `block_type == "EXCEPTION"`. Store as `context_index.exception_blocks`.

4c — Actor Registry (LLM Call 2)

Structural extraction (seed):

Collect all text from blocks with `block_type == "ACTOR"` and from table cells in columns headed "Role", "Actor", "Responsible Party", "Performed By". This gives a raw list of candidate actor strings. This extraction is structural — Docling's table output makes column identification reliable.

LLM deduplication:

Pass the full candidate list to the LLM with prompt template `DEDUPLICATE_ACTORS`. The LLM groups aliases into canonical names and returns the deduplicated registry.

This single LLM call handles: abbreviation resolution (HRBP → HR Business Partner), capitalization variants, role name synonyms, and partial matches. No fuzzy string matching logic is needed.

4d — Glossary (LLM Call 3, conditional)

Trigger condition: A definitions or glossary section exists in the document (a `HEADER` block whose text matches `/(glossary|definitions|terms)/i`).

If triggered: send the full text of that section to the LLM with prompt template `EXTRACT_GLOSSARY`. The LLM returns all term-definition pairs regardless of how they are formatted in the source (structured table, colon-separated, prose definition).

If no definitions section exists: skip this call entirely. The glossary will be empty, which is acceptable.

Validation Gate

Check	Failure code	Action
`actor_registry.actors` has at least 1 entry	`L4_NO_ACTORS`	Fail job

8. L5 — Block Enrichment

Responsibility

Enrich every `STEP`, `DECISION`, and `EXCEPTION` block with: resolved actor, condition scope, resolved cross-references, and resolved pronouns. A single LLM call handles the full enrichment pass.

Design Decision

The previous version separated enrichment into a forward pass (rules-based actor inheritance, condition propagation) and a backward pass (LLM for pronouns and cross-refs). This split was designed to minimize LLM usage, which is no longer a priority. The new approach sends all blocks requiring enrichment to the LLM in one call with full document context — the LLM resolves actors, conditions, cross-references, and pronouns simultaneously. This is more accurate because context is holistic, not local.

Structural position information (heading path, list depth, preceding blocks) is still included in the LLM prompt to give it the same context the rule-based passes used to exploit — now as signal for the LLM rather than as deterministic logic.

Inputs

- Block tree (L3 output)
- `ContextIndex` (L4 output), specifically: actor registry, section anchors

Outputs

- Every `STEP`, `DECISION`, `EXCEPTION` block has `resolved_actor`, `condition_scope`, `cross_refs`, `pronoun_resolution` populated
- `enrichment_version = 1` on all processed blocks

Processing

Step 1 — Prepare enrichment batch

Collect all blocks with `block_type` in `["STEP", "DECISION", "EXCEPTION"]`. For each, include:

- `block_id`, `raw_text`, `heading_path`, `list_depth`, `list_index`
- `preceding_sibling_text`: text of the immediately preceding sibling block (for actor and condition inheritance context)
- `parent_heading_text`: the heading text of the nearest `HEADER` ancestor

Also include in the prompt:

- Full actor registry (canonical names + aliases)
- Full section anchor list (for cross-reference resolution)

Step 2 — LLM Enrichment Call (LLM Call 4)

Send the full batch with prompt template `ENRICH_BLOCKS_BATCH`. The LLM returns for each block:

- `resolved_actor`: canonical actor name from the registry, or null
- `condition_scope`: any conditional guard applying to this block, or null
- `cross_refs`: list of `{ ref_text, resolved_block_id }` for any references in the block text
- `pronoun_resolved_actor`: if the block contains an ambiguous pronoun used as subject, the canonical actor it resolves to

Step 3 — Apply results and stamp version

Apply all returned enrichment fields to the corresponding blocks. Set `enrichment_version = 1` on each processed block. For any block where `resolved_actor` is still null after LLM enrichment, mark `needs_review = true` with reason `"Actor could not be resolved"`.

Validation Gate

Check	Failure code	Action
`enrichment_version == 1` on all STEP/DECISION/EXCEPTION blocks	`L5_INCOMPLETE_ENRICHMENT`	Fail job
Blocks with null `resolved_actor` / total enriched blocks < 0.15	`L5_HIGH_UNRESOLVED_ACTOR_RATE`	Set job `status = NEEDS_REVIEW`;

9. L6 — Block Atomizer

Responsibility

Decompose each enriched `STEP`, `DECISION`, and `EXCEPTION` block into one or more `AtomicUnit` objects. An atomic unit is the smallest independently executable action: one actor, one action, one optional condition, one optional output.

Definition of "Atomic"

A block is atomic if it describes exactly one action by one actor. Split into multiple units when:

- A block contains two distinct actions joined by "and", "then", or "after that"
- Multiple actors are referenced performing different actions
- A decision block implies both a routing decision and an action

Examples:

Raw block	Atomic units produced
"Manager reviews and approves or rejects the form."	Unit 1: Manager — Review form. Unit 2: Manager — Approve or reject form [Decision]
"Submit the form to HR and notify [illegible]"	Unit 1: Employee — Submit form to HR. Unit 2: Employee — Notify [illegible]
"If amount > $500, Finance Director must co-approve."	Unit 1: Finance Director — Co-approve expense claim [Condition: amount > $500]
"Employee completes the form."	Unit 1: Employee — Complete form [single unit, no split needed]

Inputs

- All blocks with `block_type` in `["STEP", "DECISION", "EXCEPTION"]` and `enrichment_version == 1`
- Actor registry

Outputs

- `AtomicUnit[]` persisted per block

Processing

Step 1 — Batching strategy

For small to medium documents (≤ 50 processable blocks): send all blocks in a single call — LLM Call 5.

For large documents (> 50 processable blocks): split by section (top-level heading groups) into two batches — LLM Calls 5 and 6. Each batch includes the actor registry for actor normalization.

Step 2 — LLM Atomization Call(s) (LLM Call 5, optionally 6)

Send each batch with prompt template `ATOMIZE_BLOCKS_BATCH`. The LLM receives the enriched block data (raw_text, resolved_actor, condition_scope) and returns a JSON structure mapping each `block_id` to an array of atomic units.

Step 3 — Post-processing and validation

For each returned AtomicUnit:

1. Validate `action` is non-null and non-empty. If empty: mark parent block `needs_review = true`.
2. Validate `actor` is present in the actor registry canonical names. If not: flag for review, keep the raw actor string as-is — do not silently discard.
3. Assign `is_start = true` for units in the block that is positionally first in the document tree among all `STEP`-typed blocks.

4. Assign `is_terminal = true` for units where the LLM flagged `is_terminal: true`, or where `raw_text` contains explicit terminal indicators: "process complete", "close the case", "file is archived", "no further action required", "end of process".

Validation Gate

Check	Failure code	Actio
Every processed block has `len(atomic_units) >= 1`	`L6_EMPTY_ATOMIZATION`	Fail job
At least 1 unit with `is_start == true`	`L6_NO_START_UNIT`	Fail job
At least 1 unit with `is_terminal == true`	`L6_NO_END_UNIT`	Fail job

10. L7 — Node Detector

Responsibility

Map each `AtomicUnit` to a `BPMNNode`. Apply BPMN semantic rules to determine node type.

This layer is structural — given the quality of atomic units produced by L6, the mapping is deterministic.

Inputs

- All `AtomicUnit` objects for the job

Outputs

- `BPMNNode[]`

Processing

TASK nodes

For each unit where `is_start == false` AND `is_terminal == false` AND parent block `block_type == "STEP"`:

- `bpmn_type = "TASK"`, `label = unit.action`, `actor = unit.actor`

START_EVENT and END_EVENT

- `is_start == true` → create a synthetic `START_EVENT` node (`label = "Start"`, `actor = null`). Also create the TASK node for the same unit.
- `is_terminal == true` → create a synthetic `END_EVENT` node (`label = "End"`, `actor = null`). Also create the TASK node.

GATEWAY nodes (from DECISION blocks)

For each unit whose parent block has `block_type == "DECISION"`:

- `bpmn_type = "GATEWAY"`

- `gateway_type`: defer to L8's LLM call — set to `null` here, populated in L8

BOUNDARY_EVENT nodes

For each block with `block_type == "EXCEPTION"`:

- `bpmn_type = "BOUNDARY_EVENT"`, `label` = first 8 words of block text
- Attach to the TASK node(s) identified in `exception_scope` (set in L5 enrichment)

Validation Gate

Check	Failure code	Action
At least 1 `START_EVENT`	`L7_NO_START_EVENT`	Fail job
At least 1 `END_EVENT`	`L7_NO_END_EVENT`	Fail job
All nodes have non-null `label`	`L7_UNLABELED_NODES`	Fail job

11. L8 — Edge Detector

Responsibility

Infer `BPMNEdge` objects connecting BPMN nodes. Also finalize `gateway_type` for all `GATEWAY` nodes (deferred from L7). This layer uses one LLM call to handle the semantically complex decisions — gateway type inference and non-obvious flow connections — on top of a structural base graph.

Inputs

- `BPMNNode[]` (L7 output)
- Block tree (enriched, with `cross_refs`)
- `ContextIndex`

Outputs

- `BPMNEdge[]`
- `gateway_type` fields populated on all `GATEWAY` nodes

Processing

Step 1 — Base sequence graph (structural)

Traverse `AtomicUnit` objects in document order. For each consecutive pair `(unitA, unitB)`, create a `SEQUENCE_FLOW` edge between their corresponding TASK nodes. This is the default graph that subsequent steps refine.

Step 2 — LLM Gateway and Edge Inference (LLM Call 7)

Send all `DECISION` blocks and all blocks containing `cross_refs` to the LLM with prompt template `INFER_GATEWAY_AND_EDGES`. The LLM returns:

For each `DECISION` block:

- `gateway_type`: `"XOR"`, `"AND"`, or `"OR"` with reasoning
- `branch_conditions`: list of `{ branch_text, condition_label, is_default }` for each outgoing path

For each block with unresolved or complex `cross_refs`:

- `override_edges`: list of `{ source_block_id, target_block_id, edge_label }` where document order is NOT the correct flow (e.g. loop-backs, forward jumps, explicit "proceed to step X" references)

Step 3 — Apply gateway structure

For each `GATEWAY` node:

- Set `gateway_type` from LLM output
- Remove document-order edges out of the gateway
- Create outgoing edges to the first TASK of each branch, with `label` set to the LLM-returned condition label
- Mark the default branch edge with `is_default = true`

Step 4 — Apply cross-ref edge overrides

For each `override_edge` returned by the LLM, replace or add edges as indicated. These handle loop-backs and explicit forward jumps.

Step 5 — Exception edges

For each `BOUNDARY_EVENT` node, create an edge to the TASK node referenced in its exception scope. If no specific target, edge points to the nearest `END_EVENT`.

Step 6 — Gateway convergence (structural)

For XOR divergence gateways: detect if multiple outgoing branches converge at the same subsequent TASK node (same `block_id` reachable from multiple branches). If yes, insert a synthetic merge `GATEWAY` node with edges from the last TASK of each branch, and an edge from the merge to the convergence TASK.

Validation Gate

Check	Failure code	Action
`START_EVENT` has at least 1 outgoing edge	`L8_DISCONNECTED_START`	Fail job
Dead-end TASK/GATEWAY nodes < 10% of total	`L8_DEAD_END_NODES`	NEEDS_REVIEW; continue
Orphan TASK/GATEWAY nodes (no incoming edge) < 10%	`L8_ORPHAN_NODES`	NEEDS_REVIEW; continue

12. L9 — DAG Resolver

Responsibility

Validate the BPMN graph for structural correctness, detect and resolve cycles, finalize lane assignments, and produce a clean graph ready for serialization. This layer is fully algorithmic — no LLM required.

Inputs

- `BPMNNode[]` and `BPMNEdge[]` (L8 output)

Outputs

- Validated, finalized `BPMNNode[]` and `BPMNEdge[]`
- `GraphMetadata`

Processing

Step 1 — Connectivity check

BFS from `START_EVENT`. Any node not reachable is flagged as orphan: `needs_review = true`. Orphans are retained in output with a visual annotation flag — not silently removed.

Step 2 — Cycle detection and handling

Run DFS to detect back-edges. For each cycle found:

- Cycle length == 1 (self-loop): convert the node to a `SUBPROCESS` with loop marker attribute
- Cycle length >= 2: wrap all nodes in the cycle into a `SUBPROCESS`. The back-edge becomes the loop-back flow inside the subprocess. One entry edge in, one exit edge out.

Step 3 — Gateway validation

- `AND` or `XOR` gateway with only 1 outgoing edge → downgrade to `TASK`
- Synthetic merge gateway with only 1 incoming edge → remove, reconnect predecessor directly to successor

Step 4 — Lane assignment

Group `TASK` nodes by `actor`. Each unique canonical actor becomes a BPMN lane. `START_EVENT`, `END_EVENT`, and gateways without actor assignments go into a `"process"` spanning lane.

Step 5 — Final graph validation

Check	Failure code	Action
Exactly 1 `START_EVENT`	`L9_MULTIPLE_START_EVENTS`	Fail job
At least 1 `END_EVENT`	`L9_NO_END_EVENT`	Fail job
All edges reference existing node IDs	`L9_DANGLING_EDGE`	Fail job

Orphan rate < 15%	`L9_HIGH_ORPHAN_RATE`	Fail job

GraphMetadata Output

```
interface GraphMetadata {
  node_count: number;
  edge_count: number;
  lane_count: number;
  cycle_count: number;
  orphan_node_count: number;
  gateway_count: { xor: number; and: number; or: number;
merge: number };
  subprocess_count: number;
}
```

13. L10 — BPMN Translation

Responsibility

Serialize the validated graph into a standards-compliant BPMN 2.0 XML file with layout coordinates, lane definitions, and all required element attributes.

Inputs

- Validated `BPMNNode[]`, `BPMNEdge[]`, `GraphMetadata` (L9 output)
- `ActorRegistry`

Outputs

- `.bpmn` file (BPMN 2.0 XML)
- `.json` report

Node Type → BPMN 2.0 Element Mapping

Internal type	BPMN 2.0 element
`START_EVENT`	`<startEvent>`
`END_EVENT`	`<endEvent>`
`TASK`	`<userTask>` (default); `<serviceTask>` if actor is a system
`GATEWAY (XOR)`	`<exclusiveGateway>`
`GATEWAY (AND)`	`<parallelGateway>`
`GATEWAY (OR)`	`<inclusiveGateway>`
Merge `GATEWAY`	`<exclusiveGateway>` (no label)

BOUNDARY_EVENT	`<boundaryEvent>` with `<errorEventDefinition>`
SUBPROCESS	`<subProcess isLooping="true">`
BPMNEdge	`<sequenceFlow>`
Actor lane	`<lane>` inside `<laneSet>`

Processing

Step 1 — Auto-layout (Sugiyama algorithm)

Compute `x`, `y`, `width`, `height` for all nodes using topological rank layering:

1. Topological sort → assign rank (layer) per node based on depth from `START_EVENT`
2. Within each rank, order nodes by lane then by horizontal position to minimize crossings
3. `y` from rank × (node_height + vertical_gap); `x` from position within rank × (node_width + horizontal_gap)

Standard dimensions: TASK 120×60, GATEWAY 50×50, START/END EVENT 36×36, BOUNDARY_EVENT 36×36. Horizontal gap: 60px. Vertical gap: 80px.

Step 2 — XML Serialization

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/
MODEL"
             xmlns:bpmndi="http://www.omg.org/spec/BPMN/
20100524/DI"
             xmlns:dc="http://www.omg.org/spec/DD/20100524/
DC"
             xmlns:di="http://www.omg.org/spec/DD/20100524/
DI"
             id="definitions_{job_id}"
             targetNamespace="http://bpmn.io/schema/bpmn">
  <process id="process_{job_id}" isExecutable="false">
    <laneSet id="laneSet_1">
      <!-- one <lane> per canonical actor -->
    </laneSet>
    <!-- flow elements -->
  </process>
  <bpmndi:BPMNDiagram>
    <bpmndi:BPMNPlane>
      <!-- BPMNShape and BPMNEdge with coordinates -->
    </bpmndi:BPMNPlane>
  </bpmndi:BPMNDiagram>
</definitions>
```

ID generation convention:

Element	ID format
Process	`process_{job_id}`
Task	`task_{node_id}`
Gateway	`gateway_{node_id}`
Start event	`start_{node_id}`
End event	`end_{node_id}`
Lane	`lane_{actor_slug}`
Sequence flow	`flow_{edge_id}`

Step 3 — Schema validation

Validate produced XML against BPMN 2.0 XSD using `lxml`. Schema errors fail the job.

Validation Gate

Check	Failure code	Action
XML validates against BPMN 2.0 XSD	`L10_INVALID_XML`	Fail job
All nodes have layout coordinates	`L10_MISSING_LAYOUT`	Fail job
Output file size > 0	`L10_EMPTY_OUTPUT`	Fail job

Output Report

```
{
  "job_id": "...",
  "sop_class": "HR_PROCESS",
  "graph_metadata": { "node_count": 24, "edge_count": 28,
"lane_count": 3 },
  "extraction_confidence": 0.91,
  "review_flags": [
    { "block_id": "...", "layer": 5, "reason": "Actor could
not be resolved" }
  ],
  "llm_call_log": [
    { "layer": 3, "template": "CLASSIFY_BLOCKS_BATCH",
"input_tokens": 1840, "output_tokens": 420, "cached":
false },
    { "layer": 4, "template": "DEDUPLICATE_ACTORS",
"input_tokens": 210, "output_tokens": 180, "cached": false }
```

```
  ],
  "output_file": "result_{job_id}.bpmn"
}
```

14. LLM Integration Guide

General Rules

1. Always use structured output / JSON schema enforcement mode. Never parse free-text responses.
2. Cache all LLM responses by a hash of the prompt input. Re-use cached responses on retry for idempotency and cost control.
3. If an LLM call returns invalid JSON after 2 retries: mark affected blocks `needs_review = true`, set a default/null value for the failed field, and continue. Never crash the pipeline.
4. Log every LLM call to `job.llm_call_log`: layer, template, token counts, latency, cached flag.
5. All prompts use a system prompt + user prompt structure.

Prompt Templates

`CLASSIFY_BLOCKS_BATCH`

Layer: L3 | **Call:** 1

System prompt:

```
You are a process document parser. Classify each block from a
business SOP document.

Block types:
- STEP: An action to be performed by a person or system.
- DECISION: A branching point where one of multiple paths is
chosen based on a condition.
- EXCEPTION: An error case, escalation path, or alternative
handling for an unusual situation.
- ACTOR: Defines a role, person, or system responsible for
part of the process.
- CONDITION: A precondition, scope statement, or requirement
applying to following steps.
- NOTE: Informational text not part of the process flow.
- UNKNOWN: Cannot be confidently classified.

Consider heading path and list structure as context for each
block. A numbered list item under a process heading is
typically a STEP. A "if/when/unless" sentence is typically a
```

DECISION or CONDITION depending on whether it implies a branch or a prerequisite. Respond with JSON only.

User prompt:

```
Document SOP class: {sop_class}

Blocks to classify:
{blocks_as_json_array}

Each block has: block_id, heading_path, list_depth,
list_index, raw_text, preceding_block_text.
```

Required output schema:

```
[
  {
    "block_id": "string",
    "block_type": "STEP | DECISION | EXCEPTION | ACTOR |
CONDITION | NOTE | UNKNOWN",
    "confidence": 0.0,
    "reasoning": "one sentence"
  }
]
```

DEDUPLICATE_ACTORS

Layer: L4 | **Call:** 2

System prompt:

```
You are given a list of actor names extracted from a business
process document. Many refer to the same role using different
names, abbreviations, or capitalizations. Group them into
canonical names with their aliases. Use the most complete,
formal form as the canonical name. Respond with JSON only.
```

User prompt:

```
Actor candidates:
{actor_candidates_as_json_array}
```

Required output schema:

```
[
  {
    "canonical_name": "string",
    "aliases": ["string"]
  }
]
```

EXTRACT_GLOSSARY

Layer: L4 | **Call:** 3 (conditional)

System prompt:

```
You extract term definitions from a definitions or glossary
section of a business document. Return all term-definition
pairs you can find, regardless of how they are formatted
(tables, colon-separated, prose sentences). Respond with JSON
only.
```

User prompt:

```
Section text:
{definitions_section_raw_text}
```

Required output schema:

```
[
  { "term": "string", "definition": "string" }
]
```

ENRICH_BLOCKS_BATCH

Layer: L5 | **Call:** 4

System prompt:

```
You are enriching blocks from a business SOP document. For
each block, determine:
1. resolved_actor: Who performs this action? Use only
canonical names from the actor registry. If the block
inherits its actor from the section heading or a preceding
block, apply that actor. If genuinely unresolvable, return
null.
2. condition_scope: Is there a conditional guard applying to
this block (stated in a preceding condition block, or
implicit from context)? Return the condition as a short
phrase, or null.
3. cross_refs: Does the block reference another section or
step? Return the reference text and the best matching
block_id from the section anchor list, or null if
unresolvable.
4. pronoun_resolved_actor: If the block uses "they", "them",
"it", "he", "she" as the subject of an action, which actor do
they refer to? Use the actor registry and surrounding
context.

Respond with JSON only.
```

User prompt:

```
Actor registry:
{actor_registry_as_json}

Section anchors:
{section_anchors_as_json}

Blocks to enrich:
{blocks_as_json_array}

Each block has: block_id, raw_text, heading_path, list_depth,
list_index, preceding_sibling_text, parent_heading_text.
```

Required output schema:

```
[
  {
    "block_id": "string",
    "resolved_actor": "canonical actor name or null",
    "condition_scope": "string or null",
    "cross_refs": [
      { "ref_text": "string", "resolved_block_id": "string or
null" }
    ],
    "pronoun_resolved_actor": "canonical actor name or null"
  }
]
```

ATOMIZE_BLOCKS_BATCH

Layer: L6 | **Calls:** 5 (and optionally 6)

System prompt:

```
You decompose enriched SOP blocks into atomic process units.
An atomic unit describes exactly ONE action performed by ONE
actor, with an optional guard condition and optional output
artifact.

Rules:
- If a block contains two distinct actions joined by "and",
"then", or "after that", split into two units.
- If multiple actors perform different actions in one block,
split one unit per actor.
- If a block describes a decision with multiple paths, the
decision itself is one unit (with is_decision: true); each
path's first action is a separate unit.
```

```
- Use only canonical actor names from the actor registry.
Never invent actor names.
- is_terminal should be true if the unit ends a process path
(final approval, archival, closure, no further action).
- Respond with JSON only.
```

User prompt:

```
Actor registry:
{actor_registry_as_json}

Blocks to atomize:
{blocks_as_json_array}

Each block has: block_id, raw_text, resolved_actor,
condition_scope.
```

Required output schema:

```
[
  {
    "block_id": "string",
    "atomic_units": [
      {
        "sequence_in_block": 0,
        "action": "string",
        "actor": "string",
        "condition": "string or null",
        "output": "string or null",
        "is_terminal": false
      }
    ]
  }
]
```

INFER_GATEWAY_AND_EDGES

Layer: L8 | **Call:** 7

System prompt:

```
You are analyzing the flow structure of a business process
extracted from a SOP document. For each decision block,
determine the gateway type and the conditions on each
outgoing branch. For cross-reference blocks, determine if the
document-order flow should be overridden with an explicit
jump or loop-back.

Gateway types:
```

```
- XOR: Exactly one branch is taken (mutually exclusive
conditions)
- AND: All branches execute in parallel
- OR: One or more branches may be taken

Respond with JSON only.
```

User prompt:

```
Decision blocks:
{decision_blocks_as_json}

Blocks with cross-references:
{cross_ref_blocks_as_json}

All available block_ids and their labels (for override edge
targets):
{block_id_label_map_as_json}
```

Required output schema:

```
{
  "gateways": [
    {
      "block_id": "string",
      "gateway_type": "XOR | AND | OR",
      "branches": [
        {
          "branch_text": "string",
          "condition_label": "string",
          "is_default": false
        }
      ]
    }
  ],
  "edge_overrides": [
    {
      "source_block_id": "string",
      "target_block_id": "string",
      "edge_label": "string or null",
      "override_reason": "loop_back | forward_jump |
explicit_reference"
    }
  ]
}
```

15. Validation Gates Reference

Layer	Gate check	Failure code	On fail
L1	Markdown length ≥ 100 chars	`L1_EMPTY_OUTPUT`	Fail job
L1	Docling conversion completes	`L1_CONVERSION_FAILED`	Fail job
L2	Tree has ≥ 1 non-root node	`L2_EMPTY_TREE`	Fail job
L2	At least 1 block with content	`L2_NO_CONTENT`	Fail job
L3	At least 1 STEP block	`L3_NO_STEPS_FOUND`	Fail job
L3	UNKNOWN rate < 20%	`L3_HIGH_UNKNOWN_RATE`	NEEDS_REVIEW
L4	Actor registry has ≥ 1 actor	`L4_NO_ACTORS`	Fail job
L5	enrichment_version == 1 on all target blocks	`L5_INCOMPLETE_ENRICHMENT`	Fail job
L5	Unresolved actor rate < 15%	`L5_HIGH_UNRESOLVED_ACTOR_RATE`	NEEDS_REVIEW
L6	Every processed block has ≥ 1	`L6_EMPTY_ATOMIZATION`	Fail job
L6	At least 1 unit with is_start == true	`L6_NO_START_UNIT`	Fail job
L6	At least 1 unit with is_terminal ==	`L6_NO_END_UNIT`	Fail job
L7	At least 1 START_EVENT	`L7_NO_START_EVENT`	Fail job
L7	At least 1 END_EVENT	`L7_NO_END_EVENT`	Fail job
L7	All nodes have non-null label	`L7_UNLABELED_NODES`	Fail job
L8	START_EVENT connected to ≥ 1	`L8_DISCONNECTED_START`	Fail job
L8	Dead-end node rate < 10%	`L8_DEAD_END_NODES`	NEEDS_REVIEW
L9	Exactly 1 START_EVENT	`L9_MULTIPLE_START_EVENTS`	Fail job
L9	At least 1 END_EVENT	`L9_NO_END_EVENT`	Fail job
L9	No dangling edges	`L9_DANGLING_EDGE`	Fail job
L9	Orphan rate < 15%	`L9_HIGH_ORPHAN_RATE`	Fail job
L10	XML validates against BPMN 2.0	`L10_INVALID_XML`	Fail job
L10	All nodes have layout coordinates	`L10_MISSING_LAYOUT`	Fail job

16. Open Questions for PM Sign-off

#	Question	Affe	Default if
1	**BPMN output dialect** — Which tooling must the output be compatible with? (Camunda, Signavio, Bizagi, bpmn.io) Each has	L10	Blocks L10 implementation
2	**Cycle handling policy** — When a cycle is detected, always wrap in subprocess, or allow back-edges for supported tools?	L9, L10	Default: subprocess (safe, universally
3	**Actor-to-lane mapping** — One lane per canonical actor, or grouped by department/team? Grouping requires a role hierarchy input.	L9, L10	Default: one lane per canonical actor

4	**Review workflow** — When job reaches `NEEDS_REVIEW`, is a human review interface in scope for this system or handled	All laye	Default: expose review flags via API
4 7	**SOP class taxonomy** — Are the 6 current classes sufficient? New classes may affect the context provided to L3 and L6 LLM calls.	L2, L3	Confirm before L2 implementation
4 6	**LLM provider and model** — Which provider and model for all 7 calls? Affects context window planning (large batches in L3/L5/L6	L3–L8	Blocks prompt finalization
4	**Partial output policy** — Should BPMN output include orphan nodes with an annotation, or omit them?	L9, L10	Default: include with
4 8	**Maximum document size** — Is there an upper bound on SOP length? A 200-page SOP may require L6 to split into 3+ batches, exceeding the 5–8 call budget. Define a page/token limit or an explicit policy for	L6, over all	Recommend: set 80-page soft limit; flag larger docs for

Document v1.1 — reflects Docling as extraction engine, LLM-primary architecture, and 5–8 call budget. All schemas, prompt templates, and validation gates are implementation-ready. Items in §16 require PM decision before the indicated layers are coded.